

배틀서버 작업보고

클라이언트 중심 처리 흐름을
서버 기준 구조로 전환하기 위한 기반 정리

현재 작업 4건 · 향후 작업 4건 · 구조 개선 방향



보고서 흐름

배틀서버 전환을 위한 작업 내용과 향후 계획

01 핵심 요약
배틀서버 전환을 위한 기반 구조 개선 방향

02 구조 개선 필요성
클라이언트 기술부채를 서버로 그대로 이전하지 않기 위한 배경

03 진행 작업 현황
현재까지 진행한 서버 / 클라이언트 구조 개선 작업

04 작업별 상세 내용
ThreadTimer, DBDispatcher, FSM / TaskScheduler, 캐릭터 구조 리팩토링

05 향후 진행 계획
전투, Bullet, Magic, Map 구조 개선 예정 작업

클라베이스 로직의 서버 이관을 위해

서버 처리 구조와 주요 클라이언트 로직을 함께 정리하고 있습니다.

구조 개선 필요성

기술부채를 그대로 서버로 옮기지 않기 위한
선행 구조 정리

문제

하드코딩 조건 처리 · 매직넘버
중복 구현 · 타입별 예외 처리 누적

위험

클라이언트 로직을 그대로 서버로 옮기면
서버에도 동일한 유지보수 부담이 누적됩니다.

현재 진행 작업

서버 이관에 필요한 기반 구조를 4개 작업으로 정리했습니다.

1

ThreadTimer

주기성 작업 분리

2

DBDispatcher

유저 단위 DB 병렬 처리

3

FSM / TaskScheduler

몬스터 AI 상태 관리 기반

4

캐릭터 구조 리팩토링

Char 중심 예외 처리 분리

향후 방향

전투 / Bullet / Magic / Map 구조 정리 후 단계적 서버 이관

기대 효과

부하 감소 · 영향 범위 축소 · 유지보수성 개선 · 이관 기반 확보

구조 개선 필요성

문제

현재 문제

하드코딩 조건 처리, 매직넘버, 중복 구현 로직, 타입별 예외 처리가 전체적으로 많이 남아 있습니다.
캐릭터, 스킬, 총알, 마법, 전투 로직은 거대한 switch / if-else 구조에 의존하는 부분이 많습니다.

위험

서버 이관 리스크

클라이언트 로직을 그대로 서버로 옮기면 서버 코드 역시 동일한 기술부채를 가져가게 됩니다.
단순 이관만으로는 안정적인 서버 베이스 전환이 어렵습니다.

대응

대응 방향

기능을 그대로 복사하지 않고, 서버 기준 구조에 맞게 역할별 모듈로 나누어 단계적으로 이관합니다.
먼저 기반 구조를 정리한 뒤 전투 로직을 확장합니다.

기술 상세보다 중요한 판단 기준은 “그대로 옮기지 않고, 구조를 먼저 정리한다”는 방향입니다.

현재까지 진행한 작업

서버 · 정진욱

2025년 1분기 적용

ThreadTimer

목적 : 주기성 작업 분리

개선 : 별도 타이머 / 스케줄러 구조

효과 : 메인 루프 부하 감소 · 정기 작업 관리

서버 · 정진욱

2025년 3분기 적용

DBDispatcher

목적 : DB 병목 해소 · 순서 보장

개선 : 유저별 큐 + 워커 스레드

효과 : 처리 지연 감소 · 분산 처리 기반

서버 · 정진욱

2025년 4분기 적용

FSM / TaskScheduler

목적 : 몬스터 AI 서버 이전 기반

개선 : FSM 상태 관리 + 주기 업데이트

효과 : 상태 전환 구조 정리

클라이언트 · 갈원태 / 정진욱

적용 시점 추후 확정

캐릭터 구조 리팩토링

목적 : Char 중심 예외 처리 개선

개선 : helper / manager / pool 분리

효과 : 서버 이관 위치 명확화

향후 진행 예정 작업

1

서버

전투 구조

CBattle 흐름에 물린 전투 종류별
생성, 셋업, 진행, 종료 처리 분리

2

클라이언트

Bullet 구조

switch / if-else 분기를 줄이고, 데
이터 주도형 처리 구조로 정리

3

클라이언트

Magic 구조

1천 개 규모 파생 클래스의 중복
구현을 외부 모듈과 base 계층으로
정리

4

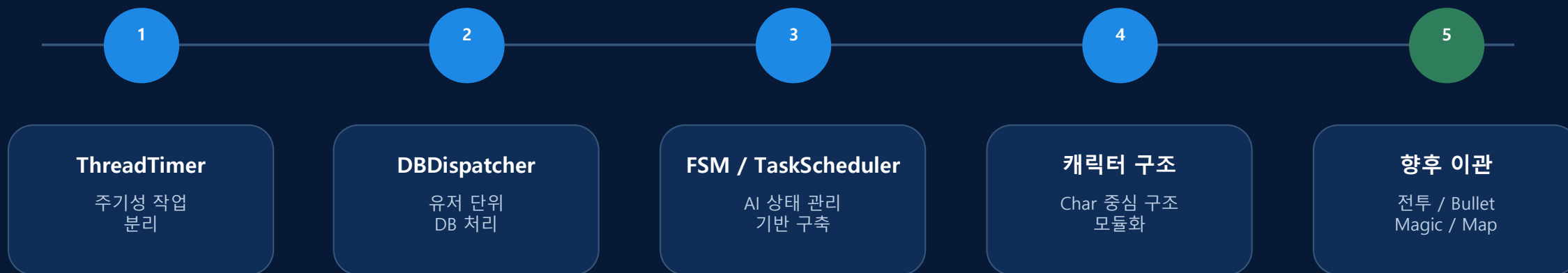
클라이언트

Map 구조

클라이언트 v1 / v2 맵 정리와 서버
용 길찾기 Map 분리

향후 작업은 클라이언트에 남아 있는 복잡한 로직을 서버 기준 구조에 맞게 정리한 뒤 단계적으로 이관하는 방향입니다.

전체 개선 로드맵



구조를 먼저 정리하고, 그 위에 서버 이관을 단계적으로 쌓아가는 흐름입니다.

ThreadTimer 시스템 개발

메인 스레드에 몰려 있던 주기성 작업을 별도 타이머 구조로 분리했습니다.

기존 구조

- 주기성 작업이 메인 루프 안에서 함께 처리
- DB 조회/갱신이 포함된 정기 작업도 메인 스레드에 영향
- 콘텐츠와 운영성 작업이 늘수록 메인 루프 복잡도 증가

개선 구조

- 시간 기반 작업을 별도 타이머/스케줄러 구조로 분리
- 타이머 ID와 핸들러 기준으로 작업 관리
- 반복 작업과 One-shot 작업을 같은 구조에서 처리

메인 루프 부하 감소

게임 진행 핵심 처리에 더 집중

정기 작업 관리 개선

일일/주간/콘텐츠 작업 일관 처리

운영성 작업 대응

신규 타이머 작업 추가 영향 축소

ThreadTimer 적용 범위

1

CThreadTimer

타이머 스레드를 관리하고, 타이머 ID에 맞는 처리 함수를 실행

2

CTimer

개별 타이머의 실행 주기, 반복 여부, 다음 실행 시점을 관리

3

CTaskScheduler

특정 시점에 실행해야 하는 예약 작업을 관리

대표 적용 작업

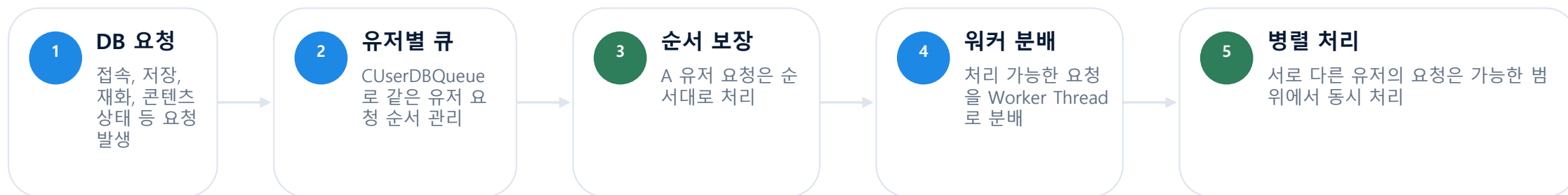
- 매시간 갱신 / 일일·주간·월간 초기화
- 콘텐츠 오픈·종료 및 랭킹 초기화
- 공성전 시작·종료·업데이트
- 상점 갱신, 반복 퀘스트 초기화, 아이템 가격 변동
- DB 조회/갱신이 포함된 정기 작업

적용 효과

- 단순 시간 체크를 넘어 서버 상태 변경 작업까지 분리
- 메인 스레드에서 직접 처리하던 부담 축소
- 반복 작업과 단발 작업을 같은 구조에서 관리
- 이후 서버 구조 개선을 이어가기 위한 기반으로 활용

DBDispatcher 시스템 개발

같은 유저의 처리 순서는 지키고, 서로 다른 유저의 요청은 병렬로 처리합니다.



순서

핵심 구조

유저별 요청은 각각의 큐에서 순서대로 처리합니다.

병렬

처리 분산

서로 다른 유저의 요청은 워커 스레드에서 병렬로 처리합니다.

효과

지연 완화

특정 유저나 콘텐츠의 요청이 전체 흐름을 막는 상황을 줄입니다.

DBDispatcher 적용 효과

중심 처리

CDBDispatcher

전체 DB 요청 처리의 중심 역할을 담당합니다. 유저 단위 요청과 공용 요청을 등록하고 워커 스레드로 분배합니다.

순서 보장

CUserDBQueue

유저별 DB 요청 큐를 관리합니다. 같은 유저의 요청이 순서대로 처리되도록 구성했습니다.

메모리 부담 감소

CDBMsg / Pool

DB 요청 메시지를 풀 기반으로 재사용합니다. 요청이 많은 상황에서 반복적인 객체 생성 부담을 줄입니다.

대표 적용 작업

- 캐릭터 정보, 아이템, 재화, 퀘스트 상태 저장
- 콘텐츠 진행 정보, 랭킹/기록 갱신
- 접속/로그아웃 관련 DB 처리와 유저별 INSERT / UPDATE 요청

기대 효과

- DB 요청이 하나의 큐나 흐름에 몰리는 문제 완화
- 전체 DB 처리 지연 감소
- 유저 데이터의 처리 순서를 지키면서 분산 처리 가능

FSM / TaskScheduler 시스템 개발

몬스터 AI를 서버로 옮기기 전에 상태 관리와 주기 업데이트 기반을 먼저 구축했습니다.



기존 문제

- 몬스터 AI가 클라이언트 코드에 있어 서버 상태 흐름 관리 기반 부족
- 상태 관리 구조가 없으면 조건문 중심으로 복잡해질 가능성
- 메인 루프 직접 처리 시 몬스터 개체 수 증가에 따른 부하 관리 어려움

개선 효과

- 몬스터 상태별 처리를 분리해 AI 로직 확장에 유리
- TaskScheduler로 FSM 업데이트를 관리 가능한 주기로 실행
- 향후 몬스터 행동 판단과 상태 전환을 서버 중심으로 처리 가능

FSM 적용 범위

구조

상태 관리

몬스터는 현재 상태를 가지고 있고, 상태에 따라 대기, 이동, 전투, 사망 등 서로 다른 처리를 수행합니다.

전환

상태 전환

상태 진입 시 처리와 상태 업데이트를 분리하고, 정해진 조건에서만 다음 상태로 변경되도록 구성했습니다.

업데이트

주기 업데이트

몬스터 AI를 매 순간 처리하지 않고, TaskScheduler를 통해 정해진 주기마다 업데이트합니다.

대표 적용 대상

- 몬스터 대기 / 이동 / 전투 / 사망 상태 처리
- 상태별 진입 및 업데이트 처리
- 클라이언트 몬스터 AI 로직의 서버 이전 준비

보고 관점

- 이 작업은 몬스터 AI 자체를 서버로 옮긴 작업이 아니라,
- 서버에서 AI 상태와 업데이트를 관리할 수 있는 기반을 먼저 만든 작업입니다.

캐릭터 구조 리팩토링

Char 중심의 예외 처리 구조를 역할별 모듈 조합 구조로 전환했습니다.

기존 구조

- Char 중심 구조에 타입별 조건 분기 누적
- CR[id] 전역 접근 증가
- 스킬별 상태 변수와 멤버 증가
- 생성/재사용/AI/패턴/소환/투사체 로직 혼재
- 기능 추가 시 수정 영향 범위 확대

개선 구조

- CharAccessor로 접근 제어
- CharacterPool / CharInstantiateModule로 생성과 재사용 분리
- 타입별 파생 클래스와 hook으로 차이 분리
- Helper / Manager / Pool 구조로 역할 분리
- AI / Pattern / Skill / Recall / Projectile / Magic 처리 모듈화

클라베이스 캐릭터 로직을 서버로 이관할 때, 각 기능이 들어갈 위치를 명확히 하기 위한 선행 작업입니다.

캐릭터 주요 분리 구조

접근

CharAccessor

CR[id] 호출 호환성을 유지하면서 내부 Char* 슬롯 배열 직접 노출을 차단

생성/재사용

CharacterPool / CharInstantiateModule

타입별 객체 생성과 재사용 책임을 별도 구조로 분리

타입별 동작

Char 파생 클래스

Hero, Mercenary, Monster, Summon 등 타입별 클래스로 분리

보조 계산

CharHelpers

타깃 탐색, 위치 계산, 인스턴스 판정 등 상태 없는 계산 로직 분리

AI / 패턴

AIDispatcher / Pattern Module

AI 판정·실행 로직과 패턴형 보스/이벤트 로직을 분리

스킬/효과

SkillDataManager / ActiveEffectTracker

스킬 상태와 구조체 접근을 Char 본체에서 분리

소환/투사체

RecallMgr / Projectile Pattern

소환 예외 처리와 발사 규칙·좌표 계산을 모듈화

메모리

CMagicPool

마법 객체 할당 정책을 풀 구조로 중앙화해 힙 비용과 단편화 완화

캐릭터 리팩토링 효과

유지보수

수정 영향 범위 축소

AI, 패턴, 패시브, 소환, 투사체, 총알 처리를 manager / module 단위로 분리해 특정 기능 수정 시 영향을 받는 범위를 줄입니다.

서버 이관

서버 이관 위치 명확화

기존처럼 Char에 상태값과 예외 처리를 계속 추가하지 않고, 기능별 구조에 맞춰 배치할 수 있습니다.

확장

확장성 개선

캐릭터 타입별 차이를 파생 클래스와 hook으로 나누어 주인공, 용병, 몬스터, 소환수 흐름을 구분해 적용하기 쉬워집니다.

스킬

스킬 상태 분리

SkillDataManager, SkillStructManager로 Char 멤버 증가 문제를 줄입니다.

메모리

메모리 관리 개선

CMagicPool로 반복적인 new/delete에 따른 비용과 단편화 가능성을 줄입니다.

구조

기반 정리

캐릭터 구조 전반을 하드코딩 분기 중심에서 모듈 조합 구조로 전환합니다.

종합 기대 효과

서버 부하 감소

메인 루프 집중 작업을 분리

처리 지연 완화

DB 요청 병목 완화

수정 범위 축소

역할별 모듈 단위 대응

서버 이관 기반

AI와 캐릭터 구조 준비

기존 접근

- 클라이언트 로직을 서버에 그대로 추가
- 메인 루프, DB, AI, Char 구조에 처리 흐름 집중
- 기능 추가 시 기존 코드 수정 범위 확대

개선 접근

- 서버 기준 구조에 맞게 역할별로 배치
- 타이머, DB, FSM, 캐릭터 모듈 구조로 분리
- 신규 콘텐츠와 전투 로직을 모듈 단위로 대응

향후 진행 방향

1

전투 종류별 처리 구조

CBattle 흐름에 물린 전투 종류별 처리를 전용 모듈 또는 파생 구조로 분리

2

Bullet 처리 구조

객체 생성, 실행 로직, 명중 이벤트를 모듈화하고 define 데이터를 데이터로 분리

3

Magic 중복 구현 정리

외부 모듈 위임과 중간 base 계층 정리로 신규 스킬 구현 부담 축소

4

Map 구조 정리

클라이언트 맵 다형성 정리와 서버용 길찾기 Map 분리

향후 작업도 복잡한 클라이언트 로직을 바로 서버로 복사하지 않고, 서버 기준 구조에 맞게 정리한 뒤 단계적으로 이관하는 방향입니다.

마무리 요약

배틀서버 작업은 단순 기능 수정이 아니라,
클라이언트 기반 로직을 서버 기준 구조로 안정적으로 옮기기 위한 기반 정리 작업
입니다.

ThreadTimer

주기성 작업 분리

DBDispatcher

DB 처리 구조 개선

FSM

AI 상태 관리 기반

캐릭터 구조

서버 이관 위치 명확화

향후

전투/Bullet/Magic/Map 정리

핵심 방향: 구조를 먼저 정리하고, 그 위에 서버 이관을 단계적으로 진행합니다.